

Exercise 6

What is a critical section? Describe in keywords: Shared resource, Concurrent access, Mutual exclusion needed, Race condition risk, Data inconsistency, Protected code block.

named and un_named semaphores: A named semaphore is a semaphore that exists in the operating system with a specific name, so multiple independent processes can access it to synchronize their actions. An unnamed semaphore exists only in the memory of a process (or in shared memory between processes) and is typically used to synchronize threads within the same process. Unlike named semaphores, unnamed semaphores are not system-wide and disappear when the memory or process ends.

Sem_t *sem = sem_open("/name", O_CREAT | O_EXCL, 0644, 1): its used to create or use an already named semaphore. As for the parameters "/name" is the name of the named semaphore and it shall always start with /. The O_CREAT, O_EXCL the first one means create if it does not exists the other means fail if it's already existing. 0644 is the access the same way as per linux. 1 is the number of resources (threads) that can use it 1 means one thread per time.

sem_unlink("/name"): its used to delete the named semaphore from the system

sem_close(sem_t *sem): to close the connection to a semaphore, the process is not a part of the shared memory, it can no longer ask for key to use the shared resources.

sem_wait(sem_t * sem): it is used to ask for a key, semaphore so it can access the shared resources. If a key is existent it returns 0 and acquires the key if the resource is still in use it returns - 1 and wait for it, as a parameter it takes the reference to the resource

sem_post(sem_t * sem): it is used to release the key, so other processes can access the shared resources. As a parameter it takes a pointer to the resource

How can you determine on a Linux system whether a named POSIX semaphore has been created?: by checking inside the directory /dev/shm/sem.* so a command line like this is going to list all the named semaphore on the system. `ls /dev/shm/sem.*`

Review the C program provided in Exercise 6.2 and describe its behavior in a few sentences. Specifically, the creation of the parent and child processes should be elaborated upon. The role of the semaphore should be described during the exercise:

The program creates two processes, a parent and a child, using `fork()`. Both processes connect to a named semaphore `mysemaphore`, which is initialized to 0. The parent immediately calls `sem_wait` and blocks because the semaphore value is 0. The child prints a message and then

calls `sem_post`, which signals the semaphore and unblocks the parent. This ensures that the parent does not continue until the child has executed, demonstrating synchronization between processes using a semaphore.

What is the purpose of the semaphore in the program: The semaphore is initialized to 0 to ensure that the parent process cannot continue past the `sem_wait` call until the child signals it by calling `sem_post`. This guarantees the correct execution order, making sure the parent waits for the child to finish its task before accessing the shared resources.